

SLAM (Simultaneous Localization and Mapping) — Detailed Concepts

Reference for: AI-Powered Cooperative Robots for Underground Mine Safety & Rescue (SIH)

1. The Core Idea

SLAM solves a chicken-and-egg problem: a robot dropped into an unknown environment (like a collapsed mine tunnel) needs to build a map of its surroundings — but to build an accurate map, it needs to know its own position, and to know its position, it needs a map. SLAM solves both **simultaneously**, using only the robot's own sensor data and motion.

2. The Two Core Sub-Problems

A. Localization — “Where am I?” Estimated from odometry (wheel encoders, IMU) fused with sensor observations of landmarks/features in the environment.

B. Mapping — “What does the environment look like?” Built incrementally as the robot observes new areas, using the estimated position to place sensor readings correctly in a global frame.

These are coupled: errors in localization corrupt the map, and a bad map causes localization errors — so SLAM algorithms are fundamentally about managing and correcting **uncertainty** in both simultaneously.

3. The Core Mathematical Framework

At each timestep, SLAM estimates:

- Robot pose: position + orientation (x, y, θ , or full 6-DOF for 3D)
- Map: set of landmarks or occupied/free cells

Two models drive this:

- **Motion model** — predicts new pose from previous pose + control input (wheel velocity, IMU), with added uncertainty/noise

- **Observation/sensor model** — relates sensor readings (LiDAR, camera) to expected landmark positions, also with noise

This is classically framed as a **Bayesian estimation problem** — maintaining a probability distribution over possible poses and maps, updated as new data arrives.

4. Major SLAM Algorithm Families

Approach	How it works	Strengths	Weaknesses
EKF-SLAM (Extended Kalman Filter)	Maintains one Gaussian estimate over pose + all landmarks, updated via linearized motion/sensor models	Simple, well-understood	Scales poorly ($O(n^2)$ with landmark count), struggles with non-linear environments
Particle Filter / FastSLAM	Each “particle” represents a hypothesis of the robot’s trajectory; landmarks estimated independently per particle	Handles non-linearity and multi-modal uncertainty well	Needs many particles for accuracy → computationally heavy
Graph-based SLAM (modern standard)	Poses and landmark observations become nodes/edges in a graph; solved via global optimization (minimizing total error across the whole graph)	Most accurate, handles loop closure well, scales better with sparse optimization	More complex to implement from scratch (but libraries handle this now)

Most real-world systems today (including what you’d use for this project) are **graph-based**.

5. Key Concepts to Understand Deeply

- **Loop closure**: recognizing you’ve returned to a previously visited location, which lets the algorithm correct accumulated drift across the whole map. Critical in looping mine tunnel networks.
- **Data association**: figuring out which sensor observation corresponds to which real-world landmark — a wrong match can corrupt the whole map.
- **Drift**: small errors in odometry accumulate over time/distance; SLAM’s job is to bound this via loop closures and sensor fusion.

- **Occupancy grid mapping:** representing the map as a grid of cells marked free/occupied/unknown — the standard output format for 2D ground-robot navigation (good fit for tunnel mapping).
- **Sensor fusion:** combining LiDAR/ultrasonic (geometry) with IMU (orientation) and wheel odometry (short-term motion) for more robust pose estimates than any single sensor gives.

6. Sensors Commonly Used

- **2D LiDAR** (e.g., RPLidar A1/A2) — cheap, good for tunnel-like corridor mapping
- **3D LiDAR** — richer but expensive, usually overkill for a hackathon budget
- **RGB-D camera** (e.g., Intel RealSense) — enables visual SLAM, lower cost than 3D LiDAR
- **IMU** — corrects for wheel slip, especially relevant on uneven mine floor
- **Wheel encoders** — baseline odometry

7. Tools/Libraries Relevant to This Project

- **Hector SLAM** — LiDAR-only, no odometry needed, works well for quick prototypes, but less robust to fast motion
- **RTAB-Map** — supports both LiDAR and RGB-D, has built-in loop closure, good for tunnel-like environments — **strong candidate for this project**
- **Google Cartographer** — production-grade, handles large-scale mapping with real-time loop closure
- **ORB-SLAM3** — visual SLAM (camera-only), useful to minimize hardware cost
- **SLAM Toolbox (ROS2)** — modern, actively maintained, integrates well if building on ROS2

8. Specific Challenges for the Mine-Rescue Use Case

- **Feature-poor environments:** long featureless tunnel walls make LiDAR-based loop closure harder (few distinct landmarks) — mitigate with reflective markers or multi-robot cross-referencing.
- **Dust/smoke:** degrades LiDAR and camera reliability — sensor fusion with IMU/gas sensors helps mitigate this.

- **Dynamic/collapsing structure:** the map may become invalid mid-mission if a tunnel collapses further — SLAM here needs to support **re-mapping** of previously mapped-but-now-changed areas.
- **Multi-robot SLAM:** since the project is cooperative, **map merging** is needed — each robot builds a local map, and these are fused into one global map when robots' paths overlap.

9. Fit With the Project Pitch

For the demo, RTAB-Map or SLAM Toolbox running on a simulated or real robot with 2D LiDAR + IMU is a realistic, achievable scope. For the “cooperative” differentiator, the multi-robot map-merging piece is what will make judges take notice — even a basic version (manually triggering a merge when two robots' maps overlap) demonstrates the concept well within hackathon time.